

Relatório 9 - Prática: Git e Github para Iniciantes (III)

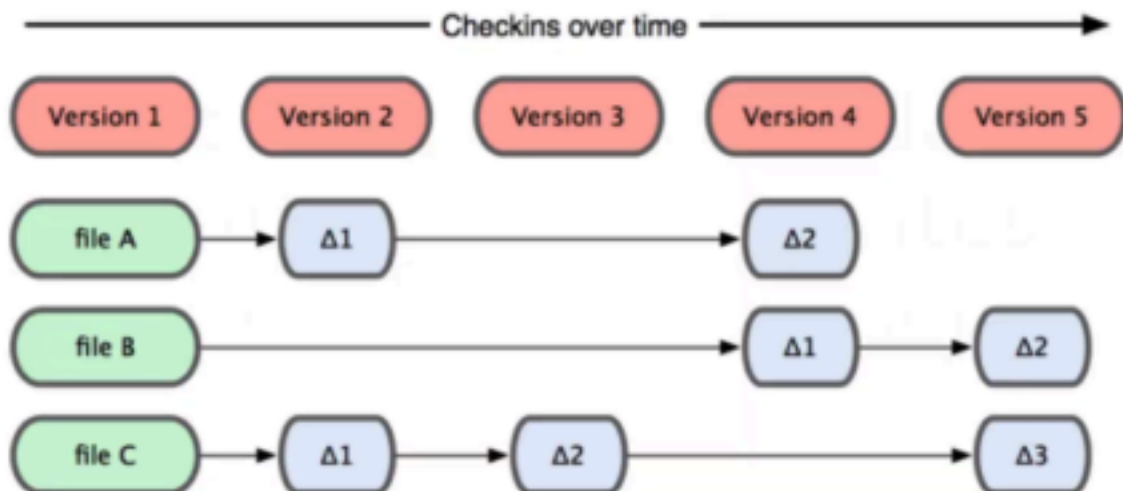
Igor Carvalho Marchi

1. Introdução

Nesta aula é apresentado dois vídeos do Youtube cobrindo todos os aspectos importantes do uso do repositório de código Github, ferramenta oficial do LAMIA. Será visto uma ferramenta que simplifica o processo de uso de linhas de comando o Github Desktop.

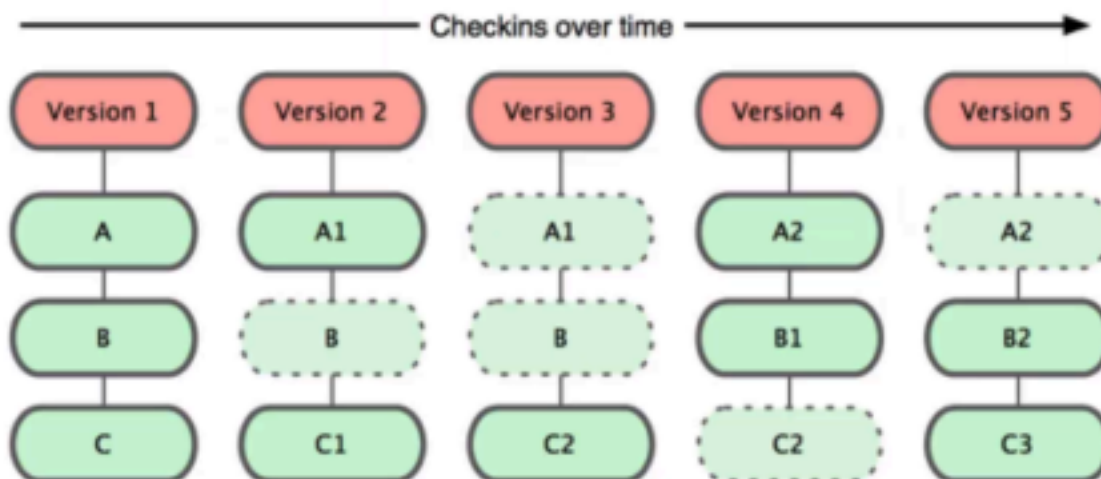
2. Desenvolvimento

O curso começa explicando a importância do git para o controle de versões, o exemplo dado é que outros controladores de versões fazem assim:



Na versão 1 foi salvo os arquivos A, B, C, porém o arquivo só vai ser salvo de novo se ele for alterado, que nem foi na versão 2 com os arquivos A e C.

Já o git age de maneira diferente:



Aqui cada versão leva as versões correspondentes dos arquivos.

A origem do git veio de uma empresa chamada BitKeeper onde eles apresentavam uma ferramenta de controle de versão que hospedava o kernel do linux, porém com alguns acontecidos o criador do Linux o Linus Torvald decidiu criar um software de controle de versões melhorado, com mais agilidade e sendo capaz de lidar com desenvolvimento não linear em projetos grandes como o próprio linux, assim nasceu o Git.

Git é a ferramenta que você instala e usa para controlar as versões do seu código, enquanto o GitHub é a plataforma online onde você armazena seus projetos e colabora com outras pessoas. Não se utiliza o GitHub sem o Git, mas o Git pode ser utilizado sem depender de qualquer plataforma de hospedagem.

Após a instalação do git ele ensina a configuração padrão, que seria a do username e do email:

```
User@DESKTOP-KB7SH6T MINGW64 ~
$ git config --global user.name "Igor Marchi"

User@DESKTOP-KB7SH6T MINGW64 ~
$ git config --global user.email "igorcmarchi@gmail.com"

User@DESKTOP-KB7SH6T MINGW64 ~
$ git config user.name
Igor Marchi

User@DESKTOP-KB7SH6T MINGW64 ~
$ mkdir git-course
```

fonte: próprio autor.

No caso ele configurou qual editor ele utiliza como padrão, mas não é necessário.

Confirmação da configuração:

Após a configuração já começa a explicação de como inicializar um repositório:

```
User@DESKTOP-KB7SH6T MINGW64 ~/git-course
$ git init
Initialized empty Git repository in C:/Users/User/git-course/.git/

User@DESKTOP-KB7SH6T MINGW64 ~/git-course (master)
$ ls -la
total 28
drwxr-xr-x 1 User 197121 0 Feb  6 10:01 ./
drwxr-xr-x 1 User 197121 0 Feb  6 10:01 ../
drwxr-xr-x 1 User 197121 0 Feb  6 10:01 .git/

User@DESKTOP-KB7SH6T MINGW64 ~/git-course (master)
$ cd .git/

User@DESKTOP-KB7SH6T MINGW64 ~/git-course/.git (GIT_DIR!)
$ ls
HEAD  config  description  hooks/  info/  objects/  refs/
```

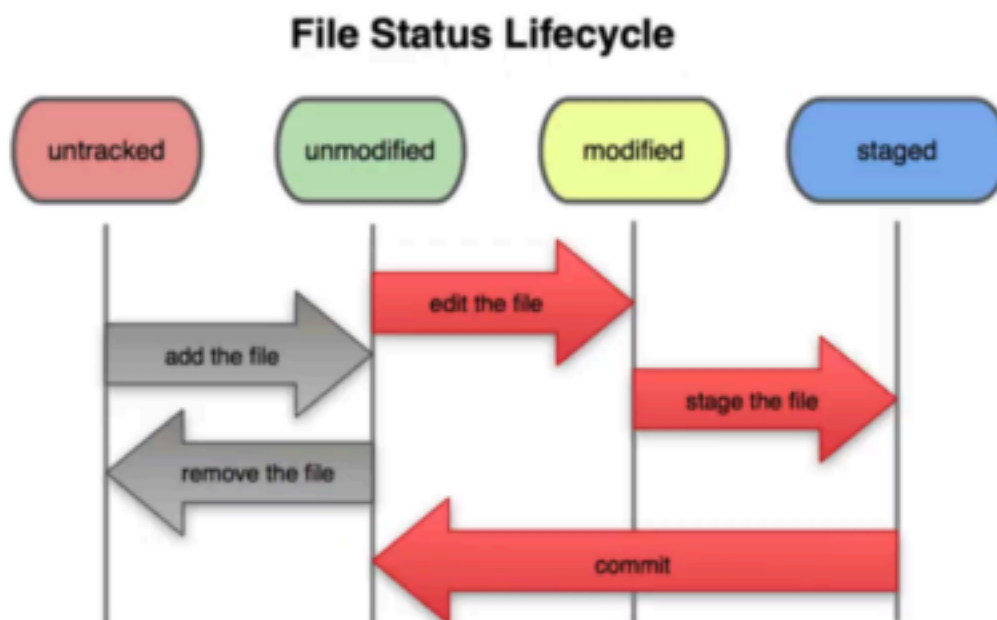
fonte: próprio autor.

Primeiramente é criada a pasta que desejamos iniciar o repositório, e entrada nela e dado o primeiro comando git 'git init', com esse comando é criado um diretório .git dentro da pasta que havia sido criada com os seguintes elementos:

Basicamente todos para a configuração e descrição do repositório.

Antes de começar a ensinar o git propriamente ele passa uma breve explicação de como usa o editor de texto dele, o vim, que é basicamente um editor de texto direto do terminal. Porém não é necessário ele deixa bem claro que se pode utilizar o editor que quiser.

Dentro do git existe o ciclo de vida dos status dos arquivos:



Untracked: É quando o arquivo foi adicionado mas ainda não foi visto pelo git.

Unmodified: Quando o git já viu ele porém não foi modificado. **Modified:** A partir de qualquer edição ele fica como modificado.

Staged: É o que ele vai manter salva a versão dos arquivos, quando você commit a todos os arquivos voltam para unmodified.

É importante ter o conhecimento desse ciclo de vida e dos logs para se ter noção do que está acontecendo com seus arquivos e as versões deles.

É possível ver os logs através do comando 'git log':

```
User@DESKTOP-KB7SH6T MINGW64 ~/git-course (master)
$ git log
commit 8d4c4ed7dda50fba15f5ae7bc6f0312128487388 (HEAD -> master)
Author: Igor Marchi <igorcmarchi@gmail.com>
Date:   Fri Feb 6 10:24:57 2026 -0300

    add link to more info

commit 49ba67b8ede2da598ddf528287bfb0741642fb52
Author: Igor Marchi <igorcmarchi@gmail.com>
Date:   Fri Feb 6 10:20:52 2026 -0300

    Add Readme.md
```

fonte: próprio autor.

Dentro do log ele mostra a hash do commit, quem foi o autor do commit e o email dele a data do commit e também a mensagem passada. Utilizando um filtro para ver apenas os commits de tal autor:

Ou mostrar os shorts logs onde filtra por apenas o nome do autor dos commits e quantos commits fizeram e as mensagens.

Além de ver o que foi mudado e comitado já é possível que se veja o que foi mudado antes de dar o commit, utilizando git diff ele consegue ver o que foi alterado dentro da pasta readme:

```
User@DESKTOP-KB7SH6T MINGW64 ~/git-course (master)
$ git diff
warning: in the working copy of 'Readme.md', LF will be replaced by CRLF the next time Git touches it
diff --git a/Readme.md b/Readme.md
index 4b60914..05dca24 100644
--- a/Readme.md
+++ b/Readme.md
@@ -3,3 +3,5 @@
 Este é um repositório para ensinar como o Git funciona.

 Saiba mais em [willianjusten.com.br](http://willianjusten.com.br)
+
+Gostou do curso? Quer mais? Ajude com uma doação, até um café é válido =)
```

fonte: próprio autor.

Importante para revisar o que foi alterado sem subir erros para a versão principal.

Com o comando 'git checkout' dá para reverter mudanças feitas ao longo do processo que não foram adicionadas ainda com o 'git add'. Porém para mudanças que já foram para o stage o que precisa ser feito é:

```
User@DESKTOP-KB75H6T MINGW64 ~/git-course (master)
$ git reset HEAD Readme.md
Unstaged changes after reset:
M   Readme.md
```

fonte: próprio autor.

Com esse 'git reset HEAD arquivo' a adição do arquivo modificado é voltada atrás. Para desfazer um commit feito existe o 'git reset' com 3 parâmetros principais: soft, mixed e hard.

Soft ele vai reverter o commit porém a mudança do arquivo vai continuar no stage.

Mixed vai reverter o commit para o status de modificado antes de ser adicionado no stage, é o parâmetro padrão.

Hard vai ignorar a existência do commit feito, uma forma meio perigosa mas útil quando foi feito algo errado.

Em uma das aulas é ensinado a como criar um repositório remoto dentro da interface do github que é bem simples apenas clicar em criar, e o vídeo seguinte é apenas como configurar dentro do github para que ele reconheça sua chave ssh para permitir que você faça commits. Com o repositório remoto criado é necessário conectar com o seu repositório local com o remoto, a própria interface do github já te ajuda com isso:

...or create a new repository on the command line

```
echo "# github-courses" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/IgorcMarchi/github-courses.git
git push -u origin main
```

...or push an existing repository from the command line

```
git remote add origin https://github.com/IgorcMarchi/github-courses.git
git branch -M main
git push -u origin main
```

fonte: próprio autor.

Se for necessário criar um repositório local ele já dá os comandos, se o repositório já existe e só precisa conectar também dá os comandos necessários ou se você vai importar código de algum outro repositório. Com a conexão feita o que é necessário para subir as alterações do repositório local para o remoto é um simples comando: 'git push':

```
User@DESKTOP-KB7SH6T MINGW64 ~/git-course (master)
$ git remote add origin https://github.com/IgorcMarchi/github-course.git

User@DESKTOP-KB7SH6T MINGW64 ~/git-course (master)
$ git push -u origin master
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 8 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (9/9), 829 bytes | 276.00 KiB/s, done.
Total 9 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), done.
To https://github.com/IgorcMarchi/github-course.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.

User@DESKTOP-KB7SH6T MINGW64 ~/git-course (master)
$ vim Readme.md

User@DESKTOP-KB7SH6T MINGW64 ~/git-course (master)
$ git commit -am "Add continuando repositório"
[master 1b4b281] Add continuando repositório
1 file changed, 2 insertions(+)

User@DESKTOP-KB7SH6T MINGW64 ~/git-course (master)
$ git push origin master
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 330 bytes | 330.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/IgorcMarchi/github-course.git
 10c8829..1b4b281  master -> master
```

fonte: próprio autor.

Esse 'origin' é um nome que fica normalmente como padrão, e esse master é de onde o arquivo está vindo que é a branch atual. Com o github é possível você pegar repositórios remotos públicos e clonar na sua máquina:

```
User@DESKTOP-KB7SH6T MINGW64 ~
$ git clone git@github.com:IgorcMarchi/github-course.git github-course-clone
Cloning into 'github-course-clone'...
remote: Enumerating objects: 12, done.
remote: Counting objects: 100% (12/12), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 12 (delta 3), reused 12 (delta 3), pack-reused 0 (from 0)
Receiving objects: 100% (12/12), done.
Resolving deltas: 100% (3/3), done.

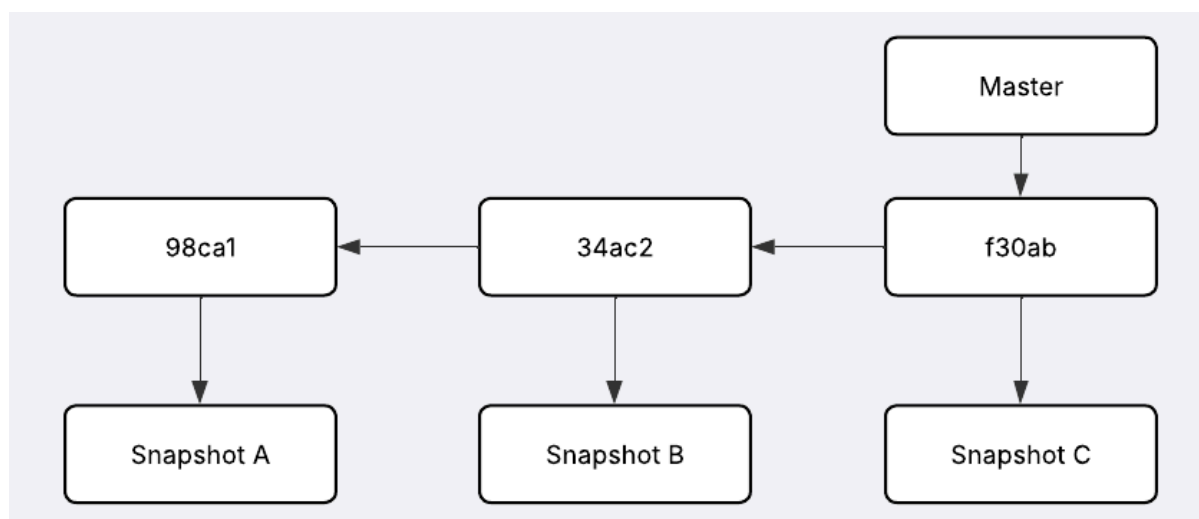
User@DESKTOP-KB7SH6T MINGW64 ~
$ cd github-course-clone

User@DESKTOP-KB7SH6T MINGW64 ~/github-course-clone (master)
$ ls
Readme.md
```

fonte: próprio autor.

O github também permite que se faça um fork de um repositório, seria parecido com um clone, porém você pode fazer as alterações que quiser e mandar um pull request para o dono do repositório, muito útil para projetos open source onde mais de uma pessoa trabalha no código. Ele se diferencia do clone porque o clone não é criado um repositório para isso e sim apenas copiado e colado os arquivos para

sua máquina local.



fonte: próprio autor.

O que uma branch faz é apontar para um commit, basicamente é isso, porém ele traz inúmeras vantagens, como poder modificar sem alterar o local principal (branch master), outro ponto é que é facilmente apagável, e é uma ferramenta primordial quando se tem várias pessoas trabalhando em um mesmo projeto evitando conflitos de código. Para criar uma nova branch é simples, basta dar 'git checkout -b {nome}' com esse comando a branch é criada e você já é levado até lá:

```

User@DESKTOP-KB7SH6T MINGW64 ~/github-course (master)
$ git checkout -b testing
Switched to a new branch 'testing'

User@DESKTOP-KB7SH6T MINGW64 ~/github-course (testing)
$ git branch
  master
* testing
  
```

fonte: próprio autor.

com 'git branch' o git vai listar todas as branches dentro do seu repositório, e vai destacar com um '*' a branch onde você está. Para alterar sua branch é simples, basta dar o mesmo comando que foi dado para criar só que sem o '-b':

```

User@DESKTOP-KB7SH6T MINGW64 ~/github-course (testing)
$ git checkout master
Switched to branch 'master'
Your branch is up to date with 'origin/master'.

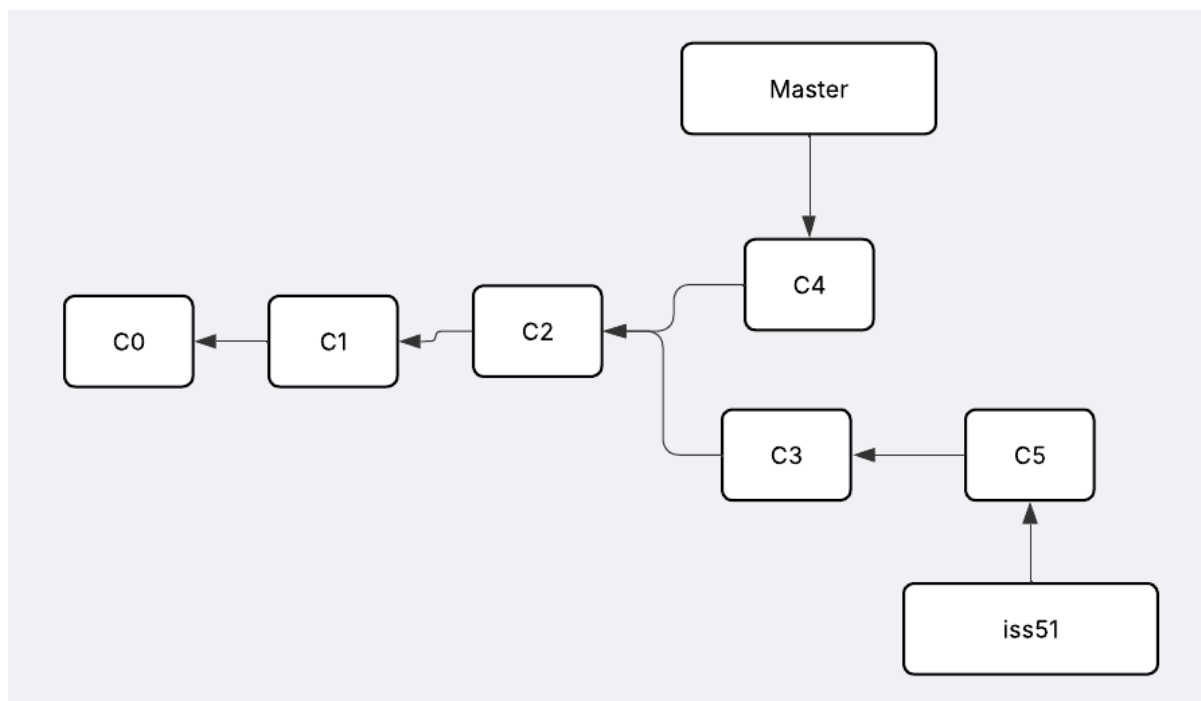
User@DESKTOP-KB7SH6T MINGW64 ~/github-course (master)
$ git branch -D testing
Deleted branch testing (was 1b4b281).

User@DESKTOP-KB7SH6T MINGW64 ~/github-course (master)
$ git branch
* master
  
```

fonte: próprio autor.

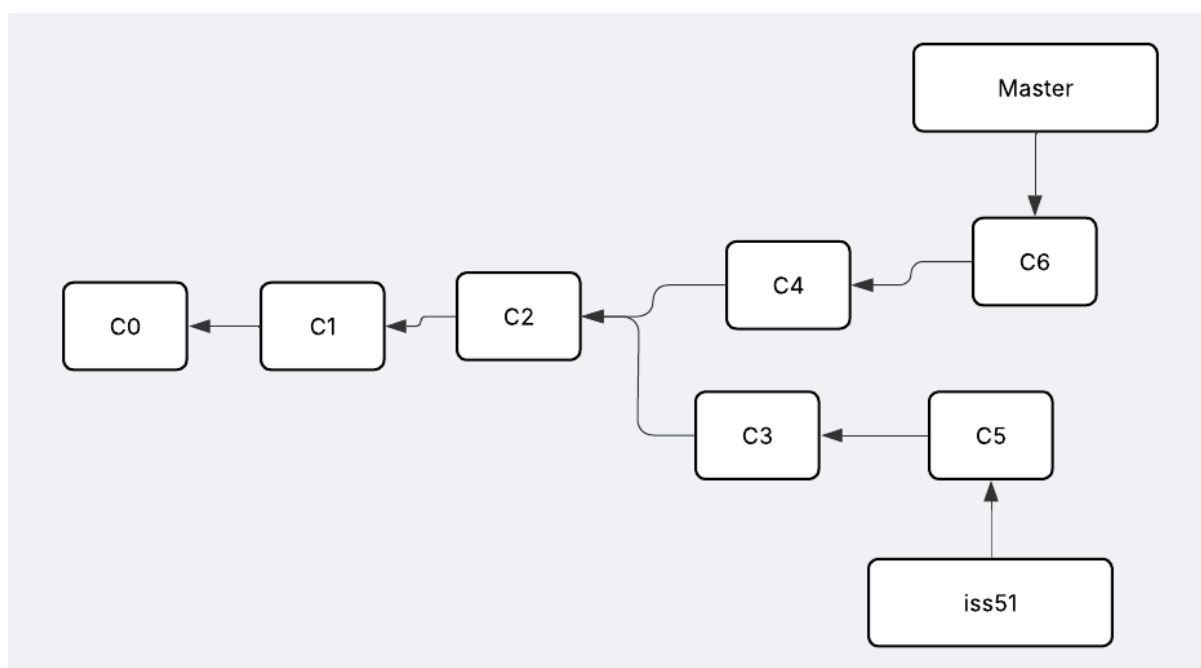
E para deletar uma branch basta dar 'git branch -D {nome}'.

Porém é necessário unir as branches para que o projeto seja finalizado e para isso dá para utilizar o dois comandos, o merge e o rebase. Começando pelo merge. Primeiramente vou mostrar o que acontece com as branches quando vão dando commits, as ramificações:



fonte: próprio autor.

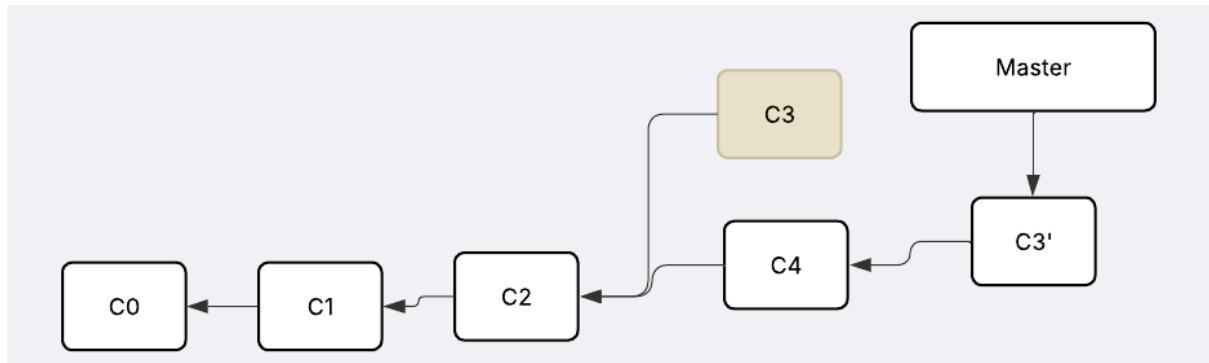
O merge vai trazer de volta a linearidade:



fonte: próprio autor.

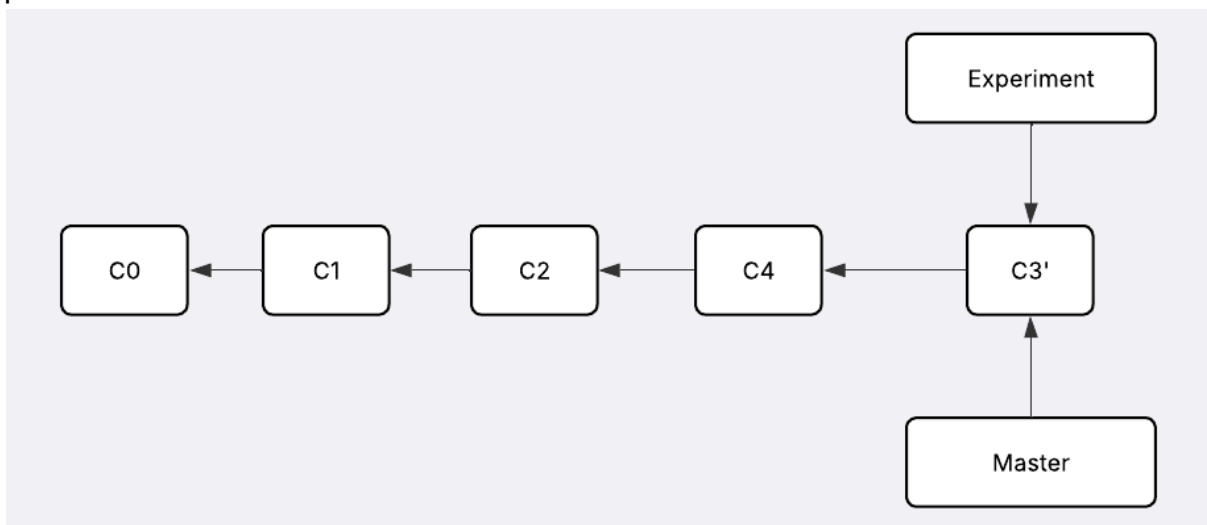
Porém é preciso fazer um commit na master que faz a junção de tudo novamente. A vantagem principal do merge é que ele não é destrutivo, não vai apagar nada que foi feito em nenhuma das branches, porém ele gera um commit a mais o que pode ser visto como poluir o histórico.

O rebase tem o mesmo intuito porém tem um processo diferente:



fonte: próprio autor.

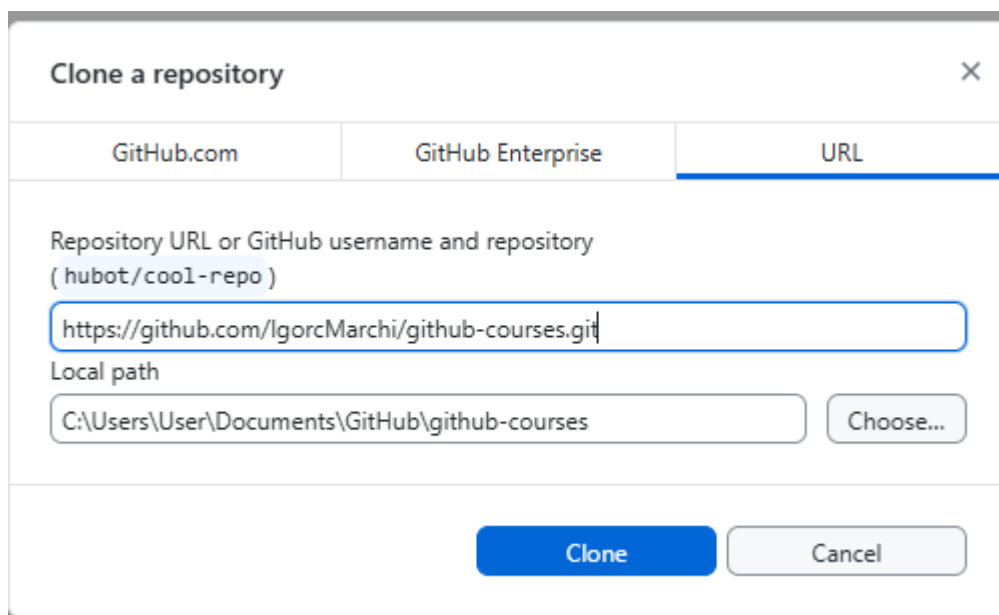
Ele basicamente destrói o commit c3 e cria uma cópia dele e coloca no começo da fila para voltar a ser linear. E no final do processo ele faz as branches apontarem para o mesmo commit:



fonte: próprio autor.

As vantagens do rebase é não ter aquele commit extra para juntar tudo, e mantém tudo de forma linear as desvantagens é que ele perde a ordem em que foi inserido os commits.

No vídeo de como utilizar o github desktop é começado apresentando a interface do programa e o que cada fração representa ali, como o histórico dos commits e as mudanças feitas onde clica para dar push para o github. Após isso ele cria um novo repositório remoto e copia o link do mesmo:



fonte: próprio autor.

Ele clona o repositório remoto através do link e define a pasta local. E ensinado a criar o .gitignore para que faça com que o git ignore todos os arquivos que forem colocados dentro desta pasta, muito útil para repositórios públicos que possam conter informações que não podem ser vazadas. Após clonar o repositório é mostrado como ele mostra o que foi mudado dentro da pasta e como fazer o commit e subir para o github e como fica o histórico das mudanças.

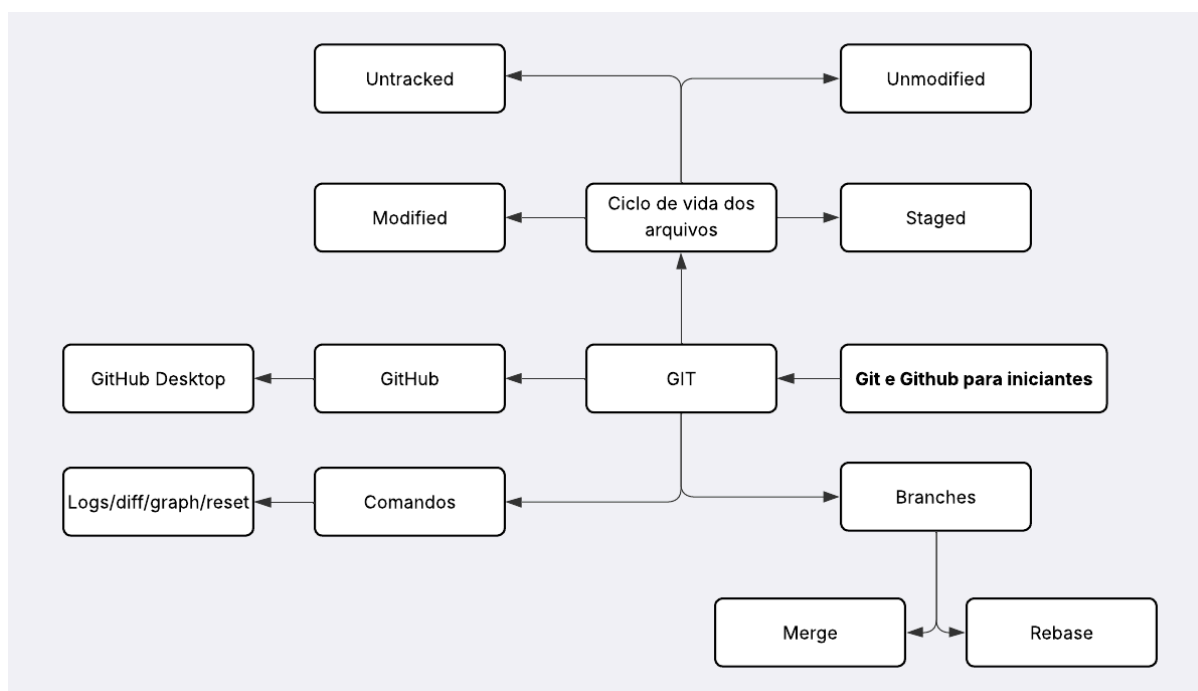


Imagem - Insight visual original

Conclusão

Após aplicar a aula é ensinado que o Git, em conjunto com o GitHub, são ferramentas extremamente poderosas para o controle de versão de códigos. O domínio dessas ferramentas é fundamental não apenas para evitar confusões entre branches e commits, mas também para compreender exatamente o que foi feito, como foi feito e por que foi feito ao longo de grandes projetos.

Referências

<https://www.youtube.com/watch?v=Fj3gtbaF8WA>

https://docs.google.com/document/d/1rAvJ0FiWZG5fgecj4VhHqDJcgQWh_O5CrmZCw812GGU/edit?tab=t.0

Para criação de imagens - Utilizado <https://lucid.app/>.

Link do git - <https://github.com/IgorcMarchi/github-course.git>